

Capítulo 5

Implementación

Este capítulo describe la implementación del modelo de Echeveste et al. dentro del framework Scikit-NeuroMSI. La implementación abarca desde la arquitectura de software hasta los algoritmos de optimización, pasando por el modelo generativo GSM, las dinámicas de la red SSN, y los módulos de soporte desarrollados.

El código fuente se estructura siguiendo los principios de diseño del framework, lo que permite que el modelo de Echeveste se integre naturalmente con los demás modelos existentes y pueda beneficiarse de las herramientas de análisis y comparación ya implementadas. A lo largo del capítulo presentamos las decisiones de diseño tomadas, las equivalencias con el código original, y las justificaciones basadas en el artículo original. La Tabla 5.3, presentada al final del capítulo, consolida todos los parámetros del modelo como referencia técnica.

5.1. Arquitectura general de la implementación

La implementación del modelo de Echeveste se estructuró en varios módulos interconectados que separan responsabilidades siguiendo el principio de responsabilidad única. Esta organización facilita el mantenimiento, las pruebas unitarias, y permite futuras extensiones de manera modular.

5.1.1. Estructura de módulos

El código está organizado en cinco archivos principales dentro del paquete `skneuromsi`, cada uno con responsabilidades claramente delimitadas (para una visión detallada de la organización modular y las dependencias entre archivos, véase la Figura 8.1 en el Capítulo 8):

- `neural/_echeveste2020.py`: contiene la clase principal `Echeveste2020` que implementa el modelo completo, incluyendo el entrenamiento en dos etapas descrito en la Sección 3.2.5, la simulación de dinámicas neuronales según la Ecuación 3.10, y la construcción de matrices de conectividad parametrizadas según la Ecuación 3.12. Este archivo también define la clase `SSNIntegrator`, que encapsula la integración numérica de las ecuaciones diferenciales.

5.1. ARQUITECTURA GENERAL DE LA IMPLEMENTACIÓN

- `neural/_echeveste_training.py`: implementa las funciones de entrenamiento utilizando JAX para diferenciación automática. Aquí se encuentra el cálculo de momentos no lineales necesarios para la activación supralineal, la evolución temporal de los momentos mediante el método ADF, y la función de costo con sus cuatro términos según la Ecuación 3.21.
- `generative/_gsm.py`: define la clase **GSM** que implementa el modelo generativo de mezcla de escala gaussiana. Su función principal es generar los inputs externos **h** para la red SSN a partir de estímulos visuales, aplicando la transformación no lineal de la Ecuación 3.16. También permite generar estímulos sintéticos y calcular los momentos objetivo que sirven como targets durante el entrenamiento.
- `neural/_echeveste_visualization.py`: proporciona funciones de visualización especializadas para analizar las dinámicas del modelo, incluyendo potenciales de membrana, tasas de disparo, espectros de potencia, y análisis de transitorios.
- `data/echeveste_data_loader.py`: proporciona acceso a los parámetros pre-entrenados por los autores originales, permitiendo cargar configuraciones validadas sin necesidad de ejecutar el costoso proceso de optimización.

La Figura 5.1 presenta el diagrama de clases UML que ilustra las relaciones entre los componentes principales. La clase **Echeveste2020** hereda de la clase base abstracta del framework **SKNMSIMethodABC**, lo que garantiza compatibilidad con las herramientas de análisis existentes. Esta clase compone un integrador **SSNIntegrator** para resolver las ecuaciones diferenciales, utiliza el modelo generativo **GSM** durante el entrenamiento para calcular los momentos objetivo, y produce resultados en formato **NDRResult** compatible con el ecosistema del framework.

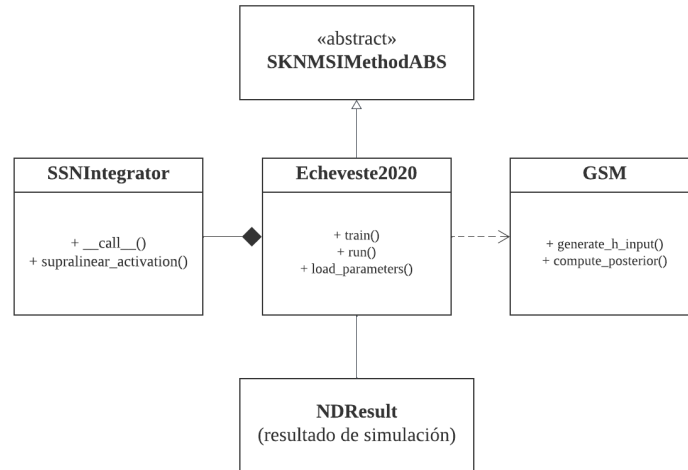


Figura 5.1: Diagrama de clases UML simplificado de la implementación del modelo de Echeveste. Para una versión detallada, véase la Figura 8.2 en el Capítulo 8.

5.1.2. Inicialización y configuración

El modelo puede inicializarse de dos maneras según las necesidades del usuario. Al pasar el parámetro `pretrained=True` durante la construcción, el modelo carga automáticamente los parámetros pre-optimizados por los autores originales, quedando listo para ejecutar simulaciones inmediatamente. Esta opción resulta conveniente para usuarios que desean explorar el comportamiento del modelo sin invertir tiempo en el proceso de optimización.

Si el modelo se inicializa con `pretrained=False` (valor por defecto), se crea con parámetros iniciales que deben ser optimizados antes de poder realizar inferencia. En este caso, el usuario debe invocar el método `train()` para ejecutar el procedimiento de optimización en dos etapas, o bien llamar al método `load_parameters()` para cargar los parámetros pre-optimizados posteriormente. Una vez configurado el modelo, el método `run()` ejecuta la simulación de las dinámicas neuronales dado un estímulo especificado, retornando un objeto `NDResult` compatible con las herramientas de análisis del framework. El flujo completo de inicialización, entrenamiento y simulación se ilustra en detalle en el diagrama de secuencia del Capítulo 8 (Figuras 8.3 y 8.4).

5.2. Implementación del modelo generativo

El modelo generativo GSM define el problema de inferencia que la red SSN debe resolver. Desde el punto de vista de la implementación, este modelo cumple dos funciones esenciales: proporciona los estímulos de entrada transformados al espacio de la red neuronal, y calcula los momentos objetivo contra los cuales se evalúa el desempeño durante el entrenamiento.

5.2.1. Clase GSM y sus responsabilidades

La clase `GSM` encapsula el modelo generativo descrito en la Sección 3.2.3. Sus responsabilidades principales incluyen la generación de filtros de Gabor que constituyen la matriz de campos proyectivos $\mathbf{A}$, la producción de parches de imagen con diferentes niveles de contraste según la Ecuación 3.4, el cálculo de la media y covarianza de la distribución posterior $P(\mathbf{y}|\mathbf{x}, z)$ según las Ecuaciones 3.18 y 3.19, y la aplicación de la transformación no lineal que convierte la proyección del estímulo en la entrada feedforward de las neuronas.

Una decisión de diseño importante fue separar la lógica del modelo generativo en una clase independiente, en lugar de integrarla directamente en la clase principal del modelo neuronal. Esta separación permite reutilizar el GSM en otros contextos, facilita las pruebas unitarias de cada componente, y refleja la distinción conceptual entre el problema computacional (definido por el GSM) y su implementación neuronal (la red SSN).

5.2.2. Transformación de entrada feedforward

Un aspecto crítico de la implementación es la transformación que convierte un parche de imagen $\mathbf{x}$ en la entrada feedforward $\mathbf{h}$ de la red neuronal, siguiendo la Ecuación 3.16. Esta transformación involucra tres parámetros optimizables ($\alpha_h, \beta_h, \gamma_h$) cuyos valores finales se presentan en la Tabla 5.3.

La implementación sigue fielmente la formulación matemática, aplicando secuencialmente la proyección lineal mediante los pesos feedforward, el desplazamiento por la línea base, la rectificación, la potenciación, y finalmente el escalado. El valor del exponente $\gamma_h \approx 2$ resulta notablemente similar al exponente supralineal $n = 2$ utilizado en la función de transferencia de las neuronas, lo que sugiere una consistencia en el procesamiento no lineal a través del modelo.

5.3. Implementación del integrador SSN

La base computacional del modelo es el integrador de la red supralineal estabilizada, que implementa las ecuaciones diferenciales que rigen la dinámica de los potenciales de membrana según la Ecuación 3.10.

5.3.1. Diseño de la clase SSNIntegrator

La clase `SSNIntegrator` se implementa como un *dataclass* de Python que encapsula las ecuaciones diferenciales del modelo. Esta decisión de diseño tiene varias ventajas: permite que el mismo integrador sea reutilizado tanto durante el entrenamiento como durante la simulación, facilita su uso con el framework BrainPy para integración numérica, y hace explícitos los parámetros del modelo como atributos inmutables de la clase.

Los atributos de la clase corresponden a las constantes temporales de las poblaciones neuronales (τ_E , τ_I , τ_η) y a los parámetros de la función de transferencia supralineal (n , k). Todos estos valores se mantienen fijos durante el entrenamiento según los valores de la Tabla 5.3.

5.3.2. Función de activación y ecuaciones dinámicas

La función de transferencia supralineal convierte los potenciales de membrana en tasas de disparo según la Ecuación 3.11. La implementación utiliza el backend matemático de BrainPy, que permite ejecución transparente en GPU cuando está disponible, proporcionando aceleración significativa para simulaciones largas.

El método principal del integrador computa las derivadas temporales de los potenciales de membrana, manejando correctamente la estructura de bloques de la matriz de conectividad. Una consideración importante en la implementación es el respeto del principio de Dale: los signos de las conexiones están incorporados directamente en la matriz $\mathbf{W}$, de modo que las conexiones desde neuronas excitatorias son siempre positivas y las conexiones desde neuronas inhibitorias son siempre negativas.

5.4. Conectividad paramétrica

Una característica distintiva del modelo de Echeveste es la parametrización circulante de la matriz de conectividad, que reduce drásticamente el número de parámetros libres respetando la simetría rotacional de la topología de anillo.

La matriz de conectividad $\mathbf{W}$ se construye a partir de solo 8 parámetros según la Ecuación 3.12. Esta parametrización reducida constituye una decisión de diseño fundamental del modelo. Con 100 neuronas en la red (50 excitatorias y

50 inhibitorias), una matriz de conectividad sin restricciones contendría 10,000 pesos sinápticos independientes. Optimizar tantos parámetros simultáneamente presentaría múltiples dificultades: el espacio de búsqueda sería enorme, el riesgo de sobreajuste a los estímulos de entrenamiento sería elevado, y el costo computacional de calcular gradientes para cada peso haría el entrenamiento impracticable.

La parametrización circulante resuelve estos problemas al explotar la simetría rotacional inherente a la topología de anillo. Dado que las neuronas están organizadas según su orientación preferida, resulta biológicamente plausible asumir que la fuerza de conexión entre dos neuronas depende únicamente de la diferencia entre sus orientaciones, no de sus valores absolutos. Esta suposición reduce drásticamente los grados de libertad: cada bloque de la matriz queda completamente determinado por solo dos parámetros, una amplitud a y un ancho d , que definen un kernel gaussiano circular.

Cada uno de los cuatro bloques de la matriz ($\mathbf{W}_{EE}$, $\mathbf{W}_{EI}$, $\mathbf{W}_{IE}$, $\mathbf{W}_{II}$) se genera aplicando este kernel a las diferencias de orientación entre neuronas presinápticas y postsinápticas. El resultado es una matriz con estructura de banda: las conexiones son más fuertes entre neuronas con orientaciones similares y decaen suavemente para orientaciones distantes. Los valores optimizados de estos parámetros se presentan en la Tabla 5.3.

La implementación construye cada bloque de manera vectorizada para mayor eficiencia computacional, evitando bucles explícitos sobre pares de neuronas. Esta vectorización resulta esencial dado que la construcción de la matriz debe realizarse múltiples veces durante el proceso de optimización, cada vez que se actualizan los parámetros de conectividad.

5.5. Proceso de entrenamiento

El entrenamiento del modelo sigue el procedimiento de optimización en dos etapas descrito en la Sección 3.2.5. La implementación utiliza JAX para diferenciación automática, combinando los optimizadores Optax (Stage 1) y SciPy (Stage 2). Para una visualización completa de las interacciones entre componentes durante ambas etapas de optimización, incluyendo el flujo de datos y las llamadas a funciones específicas, véanse las Figuras 8.3 y 8.4 en el Capítulo 8.

5.5.1. Stage 1: Optimización estocástica con ADAM

La primera etapa utiliza el optimizador ADAM con estimación estocástica de los momentos de respuesta. Los hiperparámetros de esta etapa están presentados en la siguiente tabla.

5.5. PROCESO DE ENTRENAMIENTO

Tabla 5.1: Hiperparámetros de optimización Stage 1.

Parámetro	Descripción	Valor
η	Tasa de aprendizaje ADAM	0.002
β_1	Momento de primer orden	0.9
β_2	Momento de segundo orden	0.999
N_{trial}	Trials por iteración	50
Iteraciones	Número de pasos ADAM	250

Una característica clave de Stage 1 es el “annealing” del tiempo inicial de integración $T_{\text{mín}}$, que comienza en 0 ms y aumenta progresivamente hasta $T_{\text{máx}} - 50$ ms a lo largo de las iteraciones. Este esquema de curriculum learning incentiva que la red aprenda dinámicas de muestreo rápidas desde el inicio del estímulo, evitando soluciones que solo convergen lentamente hacia los momentos objetivo.

5.5.2. Stage 2: Optimización determinística con L-BFGS-B

La segunda etapa optimiza los 15 parámetros totales utilizando el optimizador quasi-Newton L-BFGS-B con el método ADF para calcular determinísticamente los momentos de respuesta. A diferencia del método estocástico de Stage 1 que puede tener alta varianza en las estimaciones, ADF proporciona gradientes exactos (bajo la aproximación gaussiana) que permiten una convergencia más precisa.

Los parámetros adicionales optimizados en Stage 2 incluyen los 3 parámetros de la transformación de entrada ($\alpha_h, \beta_h, \gamma_h$) y los 4 parámetros de la covarianza de ruido ($\sigma_E, \sigma_I, \rho, d_\sigma$), cuyos valores óptimos se encuentran en la Tabla 5.3.

5.5.3. Función de costo

La función de costo implementa la Ecuación 3.21, penalizando diferencias entre los momentos de la distribución de respuestas de la red y los momentos objetivo del GSM. Los pesos relativos de cada término, presentados en la siguiente tabla, fueron ajustados por los autores originales para balancear la contribución de cada componente.

Tabla 5.2: Pesos de la función de costo.

Peso	Término	Valor
ϵ_μ	Error de media	$4,0 \times 10^{-5}$
ϵ_σ	Error de varianza	$8,0 \times 10^{-5}$
ϵ_Σ	Error de covarianza	$8,0 \times 10^{-7}$
ϵ_{slow}	Penalización de lentitud	$4,0 \times 10^{-10}$

El término de “slowness” penaliza la autocorrelación temporal de las respuestas, incentivando que muestras consecutivas sean estadísticamente independientes. Sin este término, la red podría aprender a producir los momentos correctos

pero con dinámicas temporales excesivamente lentas que harían el muestreo ineficiente.

5.5.4. Reimplementación en JAX

Uno de los desafíos inesperados de este trabajo fue descubrir que el código de entrenamiento original de Echeveste et al. (2020) estaba implementado en OCaml, un lenguaje de programación funcional que resulta completamente atípico para el desarrollo de modelos de redes neuronales en la actualidad. La comunidad de neurociencia computacional y aprendizaje automático ha convergido casi universalmente hacia Python como lenguaje de implementación, con frameworks como TensorFlow, PyTorch o JAX como herramientas estándar para diferenciación automática y cómputo en GPU. Encontrar una implementación en OCaml representó un obstáculo considerable que requirió una adaptación drástica de nuestra estrategia de desarrollo.

En comunicación personal con el Dr. Echeveste, pudimos comprender las razones históricas detrás de esta elección. Dado el momento en que se desarrolló el modelo original, las herramientas de diferenciación automática que hoy consideramos estándar no existían o no habían alcanzado la madurez necesaria para este tipo de aplicaciones. OCaml ofrecía entonces ciertas ventajas para implementar el método ADF, que requiere propagar gradientes a través de ecuaciones diferenciales de manera eficiente. Sin embargo, esta decisión técnica, perfectamente razonable en su contexto original, dejó el código en un estado de difícil acceso para la comunidad científica actual.

La reimplementación en JAX constituyó, por tanto, no solo una traducción de código sino una actualización tecnológica sustancial del modelo. JAX fue la elección natural por varias razones. En primer lugar, su paradigma de programación funcional (con funciones puras, sin efectos secundarios ni estado mutable) guarda cierta afinidad conceptual con OCaml, lo que facilitó la traducción de las estructuras algorítmicas del código original. Las ecuaciones del modelo pueden expresarse en JAX de forma casi idéntica a su formulación matemática, manteniendo una correspondencia directa con el paper original.

En segundo lugar, JAX ya se encontraba integrado en el framework Scikit-NeuroMSI como dependencia para otros componentes. Esta preexistencia significó que los usuarios del framework no necesitan instalar dependencias adicionales para utilizar el modelo de Echeveste, y que la implementación puede beneficiarse de utilidades y patrones ya establecidos en el código base. Esta coherencia tecnológica dentro del framework simplifica el mantenimiento a largo plazo y facilita futuras integraciones entre modelos.

Desde el punto de vista computacional, JAX ofrece compilación JIT (*just-in-time*) mediante XLA, que optimiza automáticamente las operaciones para el hardware disponible. Esto resulta particularmente beneficioso durante el entrenamiento, donde la función de costo y sus gradientes deben evaluarse cientos de veces. La transformación `jax.grad` permite obtener gradientes exactos de funciones arbitrarias de manera composicional: si una función f se compone de operaciones diferenciables, su gradiente se obtiene automáticamente sin necesidad de derivar manualmente las expresiones analíticas. Esta capacidad es esencial para el método ADF utilizado en Stage 2, donde los gradientes deben propagarse a través de la evolución temporal de los momentos de la distribución. Además, JAX proporciona soporte transparente para ejecución en GPU, permi-

tiendo acelerar significativamente tanto el entrenamiento como las simulaciones sin modificar el código.

En definitiva, la reimplementación en JAX transformó un código técnicamente correcto pero prácticamente inaccesible en una implementación moderna, mantenible y compatible con el ecosistema actual de herramientas de neurociencia computacional.

5.6. Simulación de dinámicas neuronales

Una vez entrenado el modelo (o cargados los parámetros pre-entrenados), el método `run()` ejecuta la simulación de las dinámicas neuronales. Esta sección describe los modos de simulación disponibles, el proceso de evolución temporal, y la estructura de los resultados.

5.6.1. Modos de simulación

La implementación soporta dos modos de simulación que corresponden a diferentes propósitos experimentales del trabajo original.

El **modo de fase única** (*single-phase*) mantiene el estímulo presente durante toda la simulación. Este modo replica las condiciones utilizadas durante el entrenamiento de la red, donde la función objetivo evalúa las estadísticas de muestreo en estado estacionario. Es el modo apropiado para validar que la red entrenada reproduce correctamente las distribuciones *target* del modelo GSM, y para análisis de respuestas estacionarias como la sintonización de orientación y la variabilidad neural dependiente del contraste.

El **modo de tres fases** (*three-phase*) divide la simulación en tres períodos temporales distintos: actividad espontánea pre-estímulo (225 ms por defecto), presentación del estímulo (100 ms), y actividad espontánea post-estímulo (125 ms). Este modo replica las condiciones experimentales del código de simulación original y permite estudiar fenómenos dinámicos que emergen específicamente del entrenamiento basado en muestreo: los *overshoots* transitorios al inicio del estímulo, las dinámicas de *onset/offset*, y las oscilaciones gamma moduladas por el estímulo.

El modo de tres fases soporta opcionalmente **delays por neurona** en las transiciones de estímulo, modelando la variabilidad temporal con que diferentes neuronas reciben la señal de entrada. Los delays se muestrean de una distribución normal con media de 45 ms y desviación estándar de 5 ms (valores por defecto del código original). Esta característica permite simular las condiciones utilizadas en el paper para analizar los *overshoots* promediados sobre múltiples configuraciones de delay.

La distinción entre ambos modos refleja un hallazgo central del trabajo original: aunque el entrenamiento optimiza para estado estacionario (modo *single-phase*), la red emergente exhibe dinámicas transitorias funcionalmente relevantes (modo *three-phase*). Los *overshoots* al *onset* del estímulo no son defectos sino características funcionales que mejoran la inferencia continua, compensando el sesgo introducido por promediar respuestas sobre ventanas temporales finitas (Echeveste et al., 2020).

5.6.2. Evolución temporal e integración numérica

El proceso de simulación consta de dos etapas: un período de estabilización inicial seguido del registro de actividad. La integración de las ecuaciones diferenciales (Ecuación 3.10) utiliza el método de Euler explícito con paso de tiempo $dt = 0,2$ ms, consistente con el código original. El framework BrainPy proporciona la infraestructura de integración:

```
# Initialize BrainPy ODE integrator
integrator_model = SSNIntegrator(
    tau_e=0.020, # 20 ms in seconds
    tau_i=0.010, # 10 ms in seconds
    tau_n=0.020, # 20 ms in seconds
    n=2.0, k=0.3
)
self._integrator = bp.odeint(f=integrator_model, method='euler', dt=0.2e-3)
```

Código 5.6.1: Inicialización del integrador numérico con BrainPy.

La primera etapa es el período de **burn-in**, durante el cual la red evoluciona sin registrar datos. Este período es necesario porque las condiciones iniciales (potenciales de membrana y ruido) se inicializan de manera arbitraria, y la red requiere tiempo para alcanzar el régimen estacionario estocástico desde el cual las muestras son representativas de la distribución posterior. El valor por defecto de 10,000 ms corresponde al utilizado en el código original, aunque para experimentos de transitorios se recomienda aumentarlo a 20,000 ms.

Durante tanto el burn-in como la simulación principal, el ruido de inferencia η evoluciona según un proceso de Ornstein-Uhlenbeck que mantiene la estructura de correlaciones espaciales definida por la matriz Σ_η :

```
def _update_noise(self, eta_old, L, dt, tau_n):
    """Update noise via Ornstein-Uhlenbeck process.

    (t+dt) = (t) * (1 - dt/_) + (2*dt/_) * L *
    where ~ N(0,I) and L is Cholesky factor of _ .
    """
    eps_1 = 1.0 - dt / tau_n
    eps_2 = np.sqrt(2.0 * dt / tau_n)
    xi = self._random.standard_normal(self._N)
    return eps_1 * eta_old + eps_2 * (L @ xi)
```

Código 5.6.2: Actualización del ruido correlacionado mediante proceso de Ornstein-Uhlenbeck.

El loop principal de simulación alterna entre la actualización del ruido y la integración de las dinámicas neuronales, determinando el estímulo apropiado en cada paso según el modo de simulación y la fase temporal actual.

5.6.3. Estructura del resultado

Una vez completada la evolución temporal, el método `run()` empaqueta las trayectorias registradas en un objeto `NDResult`, el formato estándar de resultados del framework Scikit-NeuroMSI. Como mencionamos en el Capítulo 4, esta elección de diseño permite que los resultados del modelo de Echeveste sean directamente compatibles con las herramientas de análisis y visualización desarrolladas para otros modelos del framework, facilitando comparaciones y análisis

unificados. Pero a su vez, la complejidad de este nuevo modelo impulso el desarrollo de herramientas nuevas para su análisis las cuales se adaptaron y trabajan bajo este contexto para una mayor coherencia general.

En el caso de querer desarrollar o entender el código de análisis nuevo, es crucial comprender en su totalidad dicho objeto `NDResult` el cual almacena cuatro series temporales que capturan completamente el estado de la red durante la simulación. La variable `excitatory_potential` contiene los potenciales de membrana $u_E(t)$ de las neuronas excitatorias, que son las variables dinámicas primarias de la Ecuación 3.10 cuyas fluctuaciones constituyen las “muestras” de la distribución posterior según la teoría de muestreo neural. La variable `inhibitory_potential` almacena los potenciales de membrana $u_I(t)$ de las neuronas inhibitorias, esenciales para estabilizar la dinámica y generar las oscilaciones gamma características. Las variables `excitatory_firing_rate` e `inhibitory_firing_rate` contienen las tasas de disparo $r_E(t)$ y $r_I(t)$ respectivamente, calculadas aplicando la función de activación supralineal (Ecuación 3.11) a los potenciales de membrana. Estas tasas son directamente comparables con registros electrofisiológicos experimentales.

Cada serie temporal tiene dimensiones $(N_{\text{neurons}}, N_{\text{timesteps}})$, donde el número de pasos temporales depende de la duración de simulación y la resolución temporal. El objeto incluye además metadatos sobre los parámetros de simulación utilizados, permitiendo reproducibilidad completa de los resultados. El siguiente fragmento ilustra cómo acceder a los datos para análisis posteriores:

```
# Ejecutar simulación
result = model.run(stimulus_contrast=0.5, simulation_time=1000.0)

# Acceder a tasas de disparo excitatorias
r_e = result.e_.excitatory_firing_rate # Shape: (50, 5000)

# Calcular media poblacional a lo largo del tiempo
mean_activity = r_e.mean(axis=0) # Shape: (5000,)

# Obtener vector de tiempos
time_vector = result.times_ # Array de 0 a 1000 ms

# Usar herramientas del framework
result.plot.heatmap(mode='excitatory_firing_rate')
stats = result.stats.summary()
```

Código 5.6.3: Acceso a los datos de simulación mediante el objeto `NDResult`.

Esta estructura unificada de resultados es particularmente relevante para el objetivo a largo plazo del proyecto: la integración del modelo de Echeveste con modelos de integración multisensorial como el de Cuppini et al. (2017). Al compartir el mismo formato de salida, ambos modelos podrán ser analizados y comparados utilizando las mismas herramientas.

5.7. Módulos de soporte implementados

Además de la implementación central del modelo de Echeveste, el presente trabajo contribuyó estructurando dos módulos completamente nuevos al framework Scikit-NeuroMSI: el módulo `data` para la gestión de datos pre-entrenados, y el módulo `generative` para modelos generativos. Estos módulos constituyen

contribuciones en la arquitectura del framework, implementación que va más allá del alcance del presente trabajo basandose en la necesidad de los modelos explicados en la Sección 5.2 y 5.6.1.

5.7.1. Módulo data: Cargadores de datos

El módulo `skneuromsi.data` fue implementado como una infraestructura completamente nueva para gestionar el acceso a datos pre-entrenados y configuraciones de modelos. Este diseño responde a la necesidad de separar claramente la lógica de carga de datos de la lógica de simulación, siguiendo el principio de responsabilidad única.

El módulo exporta dos clases principales. La clase `EchevesteDataLoader` proporciona acceso estructurado a todos los parámetros pre-entrenados del modelo de Echeveste, incluyendo la matriz de conectividad optimizada $\mathbf{W}$, la matriz de covarianza de ruido Σ_η , los parámetros de transformación de entrada $(\alpha_h, \beta_h, \gamma_h)$, y los parámetros de conectividad individuales (a_{XY}, d_{XY}) . La clase `GSMDataloader` gestiona el acceso a los componentes del modelo generativo GSM, incluyendo los filtros de Gabor pre-entrenados (matriz $\mathbf{A}$), la matriz de covarianza *a priori* $\mathbf{C}$, y los vectores de entrada h pre-calculados para diferentes niveles de contraste.

Esta arquitectura de cargadores permite que los usuarios trabajen con el modelo sin necesidad de acceder directamente al sistema de archivos, proporcionando una interfaz programática limpia y validación automática de la integridad de los datos:

```
from skneuromsi.data import EchevesteDataLoader, GSMDataloader

# Cargar parámetros de conectividad
loader = EchevesteDataLoader()
W_trained = loader.load_connectivity_matrix()
noise_cov = loader.load_noise_covariance()
input_params = loader.load_input_transformation_parameters()

# Cargar componentes del GSM
gsm_loader = GSMDataloader()
gabor_filters = gsm_loader.load_gabor_filters() # Shape: (256, 50)
prior_cov = gsm_loader.load_prior_covariance() # Shape: (50, 50)
```

Código 5.7.1: Uso de los cargadores de datos implementados.

El método `load_parameters()` de la clase `Echeveste2020` utiliza internamente el `EchevesteDataLoader` para configurar completamente el modelo con los parámetros pre-optimizados, dejándolo listo para ejecutar simulaciones inmediatamente. Esta funcionalidad resulta esencial para facilitar el uso del modelo sin necesidad de ejecutar el costoso proceso de entrenamiento.

5.7.2. Módulo generative: Modelos generativos

El módulo `skneuromsi.generative` representa una extensión conceptual significativa del framework, introduciendo por primera vez la capacidad de definir modelos generativos que especifican el problema de inferencia que las redes neuronales deben resolver. Mientras que los modelos existentes en Scikit-NeuroMSI se enfocaban exclusivamente en la integración de señales sensoriales,

este nuevo módulo permite definir distribuciones *a priori* y calcular distribuciones posteriores que sirven como objetivos de entrenamiento.

Actualmente el módulo contiene la clase **GSM** que implementa el modelo de mezcla de escala gaussiana descrito en la Sección 3.2.3. Sin embargo, la arquitectura fue diseñada con extensibilidad en mente: futuros trabajos podrían agregar otros modelos generativos (como modelos de campo aleatorio de Markov, modelos de energía, o modelos generativos multimodales) siguiendo la misma interfaz, habilitando así el estudio de cómo diferentes supuestos generativos afectan las dinámicas de muestreo neural.

5.8. Infraestructura de validación

La validación de la implementación requirió el desarrollo de una infraestructura complementaria para generar simulaciones sistemáticas y producir figuras comparativas con los resultados del artículo original. Esta infraestructura, aunque externa al paquete principal de Scikit-NeuroMSI, constituye una contribución importante del presente trabajo al permitir la reproducción rigurosa de los experimentos de Echeveste et al. (2020) y dichos resultados serán explicados en el Capítulo 6.

5.8.1. Pruebas unitarias y comparación con código original

Se implementaron pruebas unitarias para verificar el comportamiento correcto de los componentes individuales, incluyendo la inicialización con parámetros por defecto, la verificación de valores correctos en la matriz de conectividad, el comportamiento de la función supralineal, y la reproducibilidad de resultados con semillas fijas.

Aunque el código original está escrito en OCaml, se realizaron comparaciones numéricas para verificar que los filtros de Gabor generados coinciden con los archivos proporcionados, que la matriz de conectividad construida desde parámetros reproduce fielmente la matriz pre-entrenada, y que los momentos posteriores del GSM coinciden para estímulos de prueba.

5.8.2. Arquitectura de dos etapas para generación de figuras

El flujo de trabajo para reproducir las figuras del artículo original se estructuró en dos etapas claramente separadas, siguiendo buenas prácticas de ingeniería de software para análisis reproducible.

La primera etapa consiste en la **generación de datos**: scripts ubicados en el directorio **figures/data/** ejecutan simulaciones con configuraciones específicas y almacenan los resultados en formato comprimido. Esta separación permite ejecutar simulaciones computacionalmente costosas una única vez y reutilizar los datos para múltiples análisis.

La segunda etapa consiste en la **generación de figuras**: scripts en directorios **figures/FigX/** (donde X corresponde al número de figura del artículo original) cargan los datos pre-generados y producen visualizaciones comparativas entre el observador ideal (GSM) y la red neuronal (SSN).

El script principal `generate_simulations.py` ejecuta simulaciones para los cinco niveles de contraste del conjunto de entrenamiento original ($z \in \{0, 0,125, 0,25, 0,5, 1,0\}$) con una configuración unificada. Para cada nivel de contraste, el script instancia el modelo, carga los parámetros pre-entrenados, ejecuta la simulación, calcula estadísticas del estado estacionario, y almacena las trayectorias temporales y estadísticas en formato NumPy comprimido (`.npz`) junto con metadatos en formato JSON para trazabilidad.

Esta infraestructura permite no solo validar la implementación contra los resultados publicados, sino también explorar configuraciones alternativas de manera sistemática y reproducible. Los resultados detallados de estas comparaciones, incluyendo el análisis de momentos de respuesta, estructuras de correlación, y propiedades dinámicas emergentes, se presentan en el Capítulo 6.

5.9. Referencia de parámetros

A modo de referencia, la Tabla 5.3 consolida todos los parámetros del modelo descritos a lo largo de este capítulo, distinguiendo entre aquellos que permanecen fijos durante el entrenamiento y aquellos que son optimizados. Esta tabla corresponde a la Tabla Suplementaria S1 del artículo original (Echeveste et al., 2020). Los valores optimizados corresponden a los obtenidos por los autores tras el procedimiento de entrenamiento en dos etapas, y están disponibles en nuestra implementación mediante el método `load_parameters()`.

Categoría	Parámetro	Símbolo	Valor
Red neuronal (fijos)	Neuronas excitatorias	N_E	50
	Neuronas inhibitorias	N_I	50
	Constante de tiempo E	τ_E	20 ms
	Constante de tiempo I	τ_I	10 ms
	Constante de tiempo ruido	τ_η	20 ms
	Exponente supralineal	n	2.0
	Factor de escala	k	0.3
Modelo GSM (fijos)	Número de píxeles	N_x	256
	Número de orientaciones	N_y	50
	Desv. estándar ruido píxel	σ_x	10.0
	Factor escala $\mathbf{W}^{ff}$	—	1/15
Conectividad (optimizados)	Amplitud E→E	a_{EE}	1.46
	Amplitud E→I	a_{EI}	1.29
	Amplitud I→E	a_{IE}	−1.96
	Amplitud I→I	a_{II}	−1.21
	Ancho E→E	d_{EE}	0.56
	Ancho E→I	d_{EI}	0.56
	Ancho I→E	d_{IE}	0.40
	Ancho I→I	d_{II}	0.40
Covarianza ruido (optimizados)	Desv. estándar ruido E	σ_E	2.80
	Desv. estándar ruido I	σ_I	1.60
	Correlación E-I	ρ	0.75
	Ancho correlación	d_σ	0.45
Entrada (optimizados)	Escala entrada	α_h	1.96
	Línea base entrada	β_h	0.10
	Exponente entrada	γ_h	2.03

Tabla 5.3: Parámetros completos del modelo de Echeveste et al., correspondientes a la Tabla Suplementaria S1 del artículo original.

5.10. Resumen del capítulo

Este capítulo presentó la implementación completa del modelo de Echeveste et al. dentro del framework Scikit-NeuroMSI, junto con la infraestructura de soporte necesaria para su validación. Los principales aportes incluyen una arquitectura modular que separa claramente el modelo generativo (**GSM**), el integrador de dinámicas (**SSNIntegrator**), y la clase principal (**Echeveste2020**); la implementación del procedimiento de optimización en dos etapas usando ADAM y L-BFGS-B con ADF; y la compatibilidad total con el framework mediante herencia de la clase base abstracta.

Como contribuciones adicionales al framework, se desarrolló el módulo **data** con una nueva infraestructura de cargadores que proporciona acceso estructurado a los parámetros pre-entrenados y componentes del modelo generativo, y el módulo **generative** que introduce por primera vez modelos generativos como el GSM, permitiendo definir distribuciones *a priori* y calcular los momentos

posteriores que sirven como objetivos de entrenamiento. Este último módulo fue diseñado con extensibilidad en mente para incorporar futuros modelos generativos.

La reimplementación en JAX transformó el código original en OCaml en una implementación moderna con soporte para aceleración en GPU. La infraestructura de validación desarrollada, basada en un sistema de dos etapas para generación de datos y visualización, permite ejecutar simulaciones sistemáticas y producir figuras comparativas de manera reproducible. Los resultados detallados de estas comparaciones se presentan en el Capítulo 6, donde se analiza la concordancia entre los momentos de respuesta de la red y los objetivos del modelo generativo, las estructuras de correlación, y las propiedades dinámicas emergentes.

Capítulo 6

Resultados

Completar: RESULTADOS

Capítulo 7

Conclusiones y trabajo a futuro

Completar: CONCLUSIONES

Capítulo 8

Figuras complementarias

Este apéndice contiene diagramas técnicos de mayor detalle que complementan las explicaciones del Capítulo 5. Estas figuras proporcionan una visión más exhaustiva de la arquitectura de software y los flujos de ejecución, resultando útiles para lectores interesados en comprender o extender la implementación.

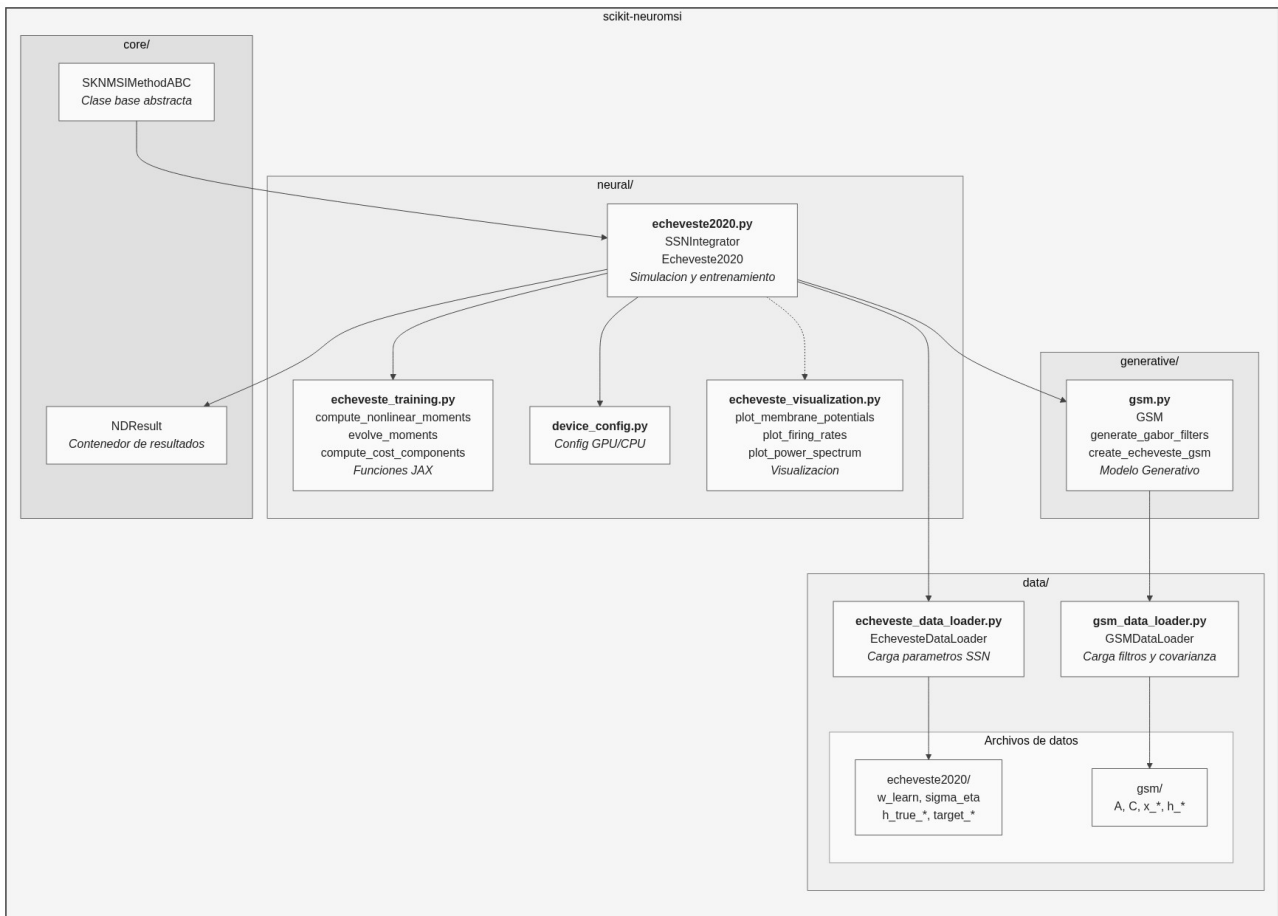


Figura 8.1: Estructura modular del paquete `scikit-neuromsi`. Se detalla la organización del código en submódulos especializados: el núcleo (**core/**) con la clase base abstracta y el contenedor de resultados, el módulo neuronal (**neural/**) con las clases de simulación y las funciones de entrenamiento en JAX, el módulo generativo (**generative/**) con el modelo GSM, y el sistema de gestión de datos (**data/**) con los cargadores especializados.

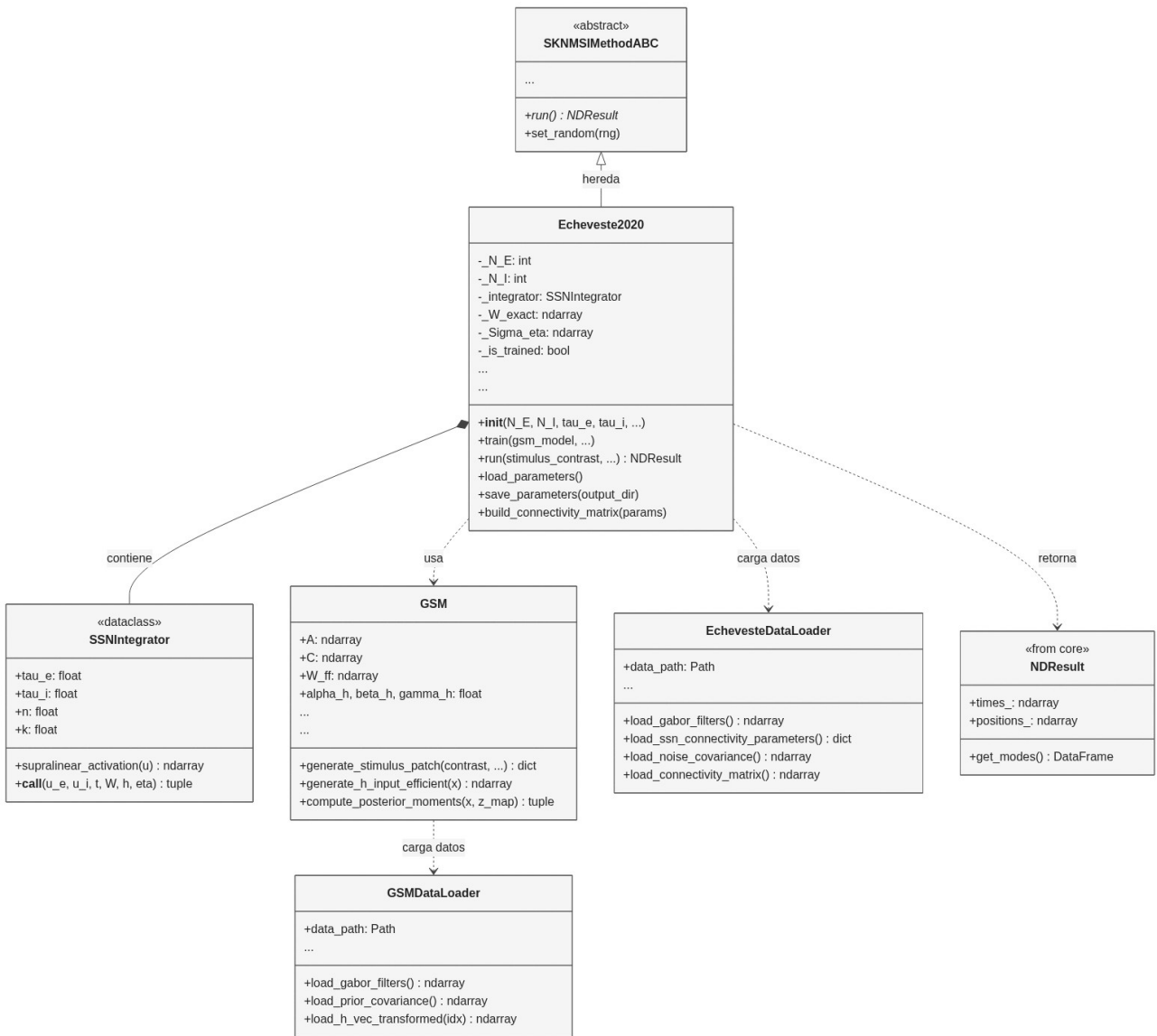


Figura 8.2: Diagrama de clases UML detallado de la implementación Echeveste2020. Se observa la herencia desde la clase base abstracta **SKNMSIMethodABC**, la composición del integrador **SSNIntegrator** como dataclass, las dependencias con el modelo generativo **GSM** y los cargadores de datos (**EchevesteDataLoader**, **GSMDDataLoader**) para la configuración de la red. Los parámetros de los modelos están simplificados indicándolo con "...".

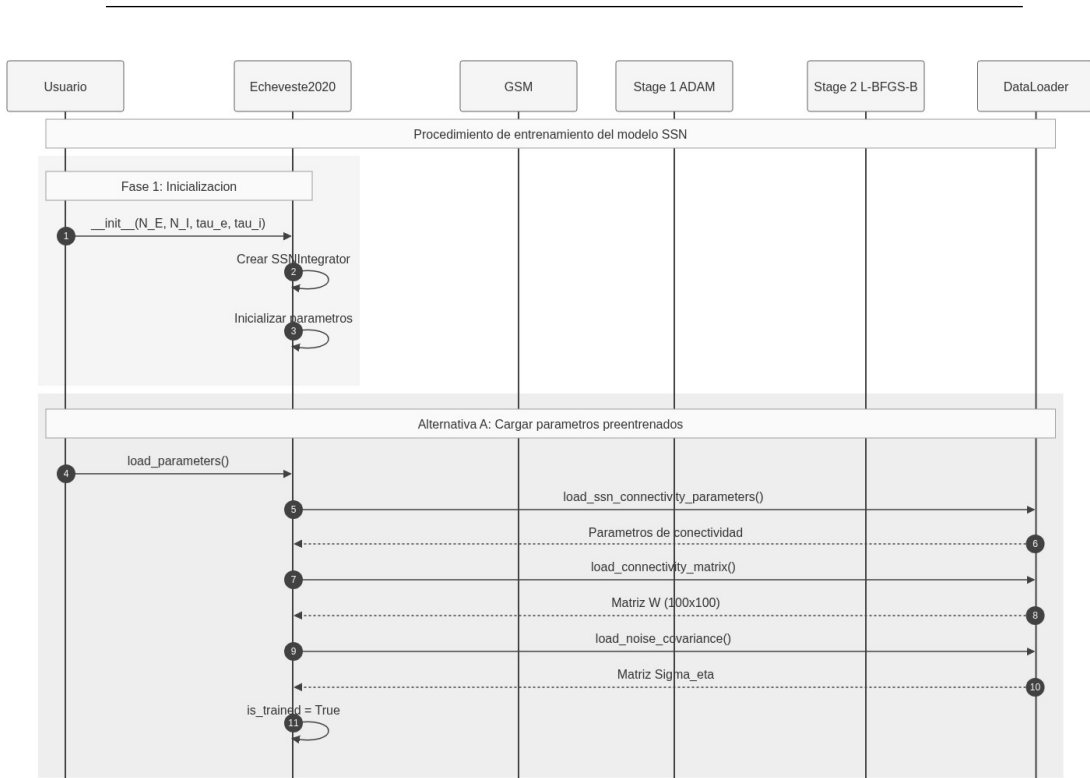


Figura 8.3: Diagrama de secuencia del flujo de ejecución (parte 1 de 2). Muestra la Fase 1 de inicialización del modelo con sus parámetros, seguida de la Alternativa A que ilustra el proceso de carga de parámetros pre-entrenados mediante el **DataLoader**, incluyendo la obtención de la matriz de conectividad $\mathbf{W}$ y la matriz de covarianza de ruido Σ_{η} .

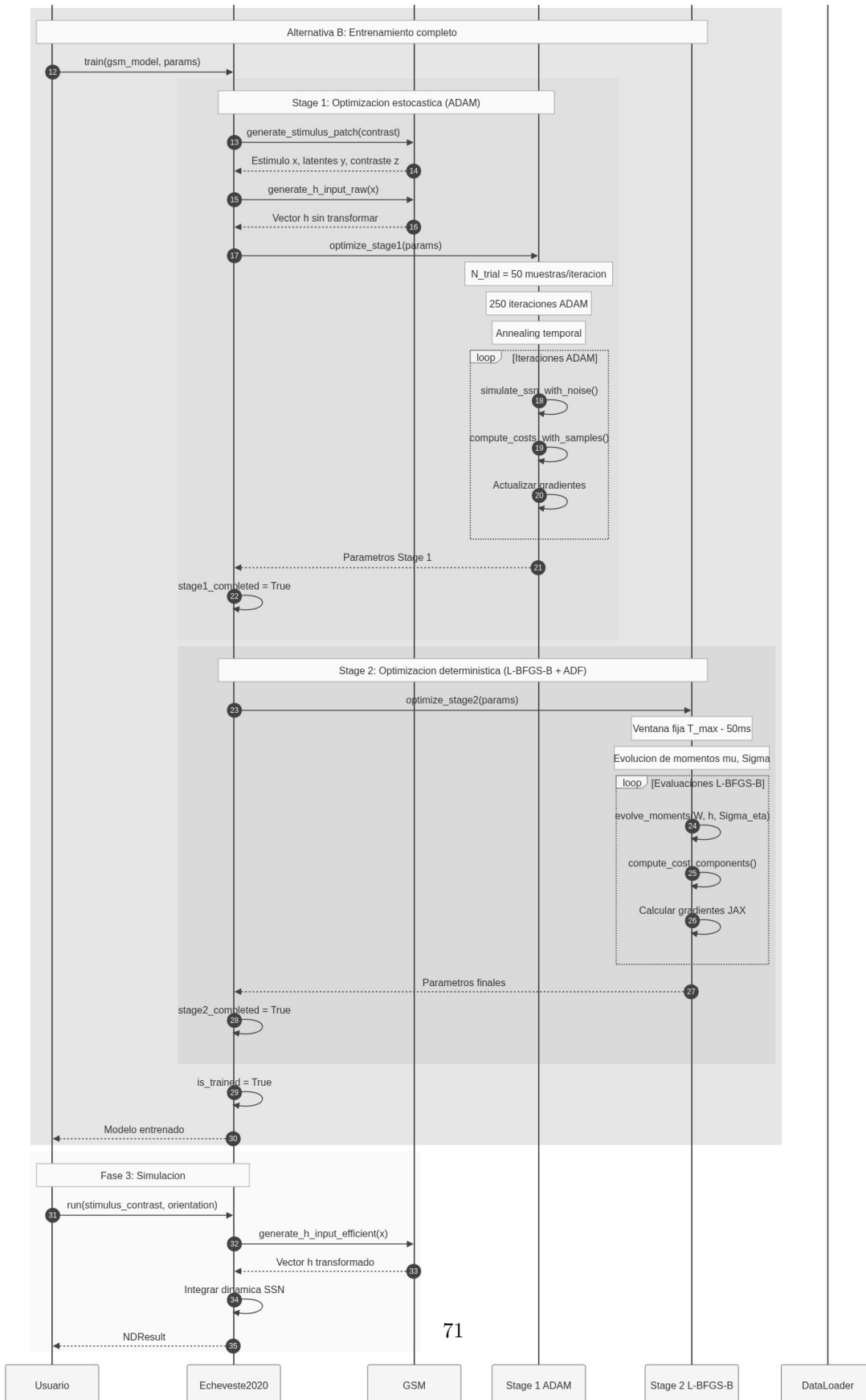


Figura 8.4: Diagrama de secuencia del flujo de ejecución (parte 2 de 2). Muestra la Alternativa B correspondiente al entrenamiento completo y se integra la dinámica SSN para producir el `NDRResult`.

Bibliografía

- Alais, D. and Burr, D. (2004). The ventriloquist effect results from near-optimal bimodal integration. *Current biology*, 14(3):257–262.
- Bishop, C. M. and Nasrabadi, N. M. (2006). *Pattern recognition and machine learning*, volume 4. Springer.
- Blanco, C. (2014). *Historia de la neurociencia*. Biblioteca nueva Madrid.
- Cuppini, C., Shams, L., Magosso, E., and Ursino, M. (2017). A biologically inspired neurocomputational model for audiovisual integration and causal inference. *European Journal of Neuroscience*, 46(9):2481–2498.
- Dale, H. (1935). Pharmacology and nerve-endings.
- Dayan, P. and Abbott, L. F. (2005). *Theoretical neuroscience: computational and mathematical modeling of neural systems*. MIT press.
- Echeveste, R., Aitchison, L., Hennequin, G., and Lengyel, M. (2020). Cortical-like dynamics in recurrent circuits optimized for sampling-based probabilistic inference. *Nature neuroscience*, 23(9):1138–1149.
- Eliasmith, C. and Anderson, C. H. (2003). *Neural engineering: Computation, representation, and dynamics in neurobiological systems*. MIT press.
- Gerstner, W. and Kistler, W. M. (2002). *Spiking neuron models: Single neurons, populations, plasticity*. Cambridge university press.
- Heaton, J. (2018). Ian goodfellow, yoshua bengio, and aaron courville: Deep learning: The mit press, 2016, 800 pp, isbn: 0262035618. *Genetic programming and evolvable machines*, 19(1):305–307.
- Hodgkin, A. L. and Huxley, A. F. (1952). A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of physiology*, 117(4):500.
- Hubel, D. H. and Wiesel, T. N. (1959). Receptive fields of single neurones in the cat’s striate cortex. *The Journal of Physiology*, 148(3):574–591.
- Körding, K. P., Beierholm, U., Ma, W. J., Quartz, S., Tenenbaum, J. B., and Shams, L. (2007). Causal inference in multisensory perception. *PLoS one*, 2(9):e943.
- Marr, D. (2010). *Vision: A computational investigation into the human representation and processing of visual information*. MIT press.

BIBLIOGRAFÍA

- Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill, New York.
- Murphy, K. P. (2012). *Machine learning: a probabilistic perspective*. MIT press.
- Nobel Prize Committee (2024). The Nobel Prize in Physics 2024. <https://www.nobelprize.org/prizes/physics/2024/summary/>. Accessed: 2025.
- Paredes, R., Cabral, J. B., and Seriès, P. (2025). Scikit-neuromsi: A generalized framework for modeling multisensory integration. *Neuroinformatics*, 23(3):40.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.
- Stein, B. E. (2012). *The new handbook of multisensory processing*. Mit Press.
- Willis, T. (1664). *Cerebri anatome*, volume 1. Apud Gerbrandum Schagen.
- Yamins, D. L., Hong, H., Cadieu, C. F., Solomon, E. A., Seibert, D., and DiCarlo, J. J. (2014). Performance-optimized hierarchical models predict neural responses in higher visual cortex. *Proceedings of the national academy of sciences*, 111(23):8619–8624.